



Capítulo 3

Control de Flujo y Validaciones

Objetivos:

- Explicar cómo funciona la estructura condicional IF / ELSE IF / ELSE en Robot Framework y en qué contextos se aplica.
- Describir las diferencias sintácticas entre el bloque IF nativo de RF y el enfoque heredado con Run Keyword If.
- Interpretar el flujo de ejecución de un test case o keyword que contiene ramas condicionales con múltiples condiciones.
- Las condiciones se evalúan como expresiones Python, lo que permite usar operadores de comparación (==, !=, >, <) y lógicos (and, or, not).

3.1: IF / ELSE IF / ELSE: Ejecución Condicional en Keywords y Test Cases

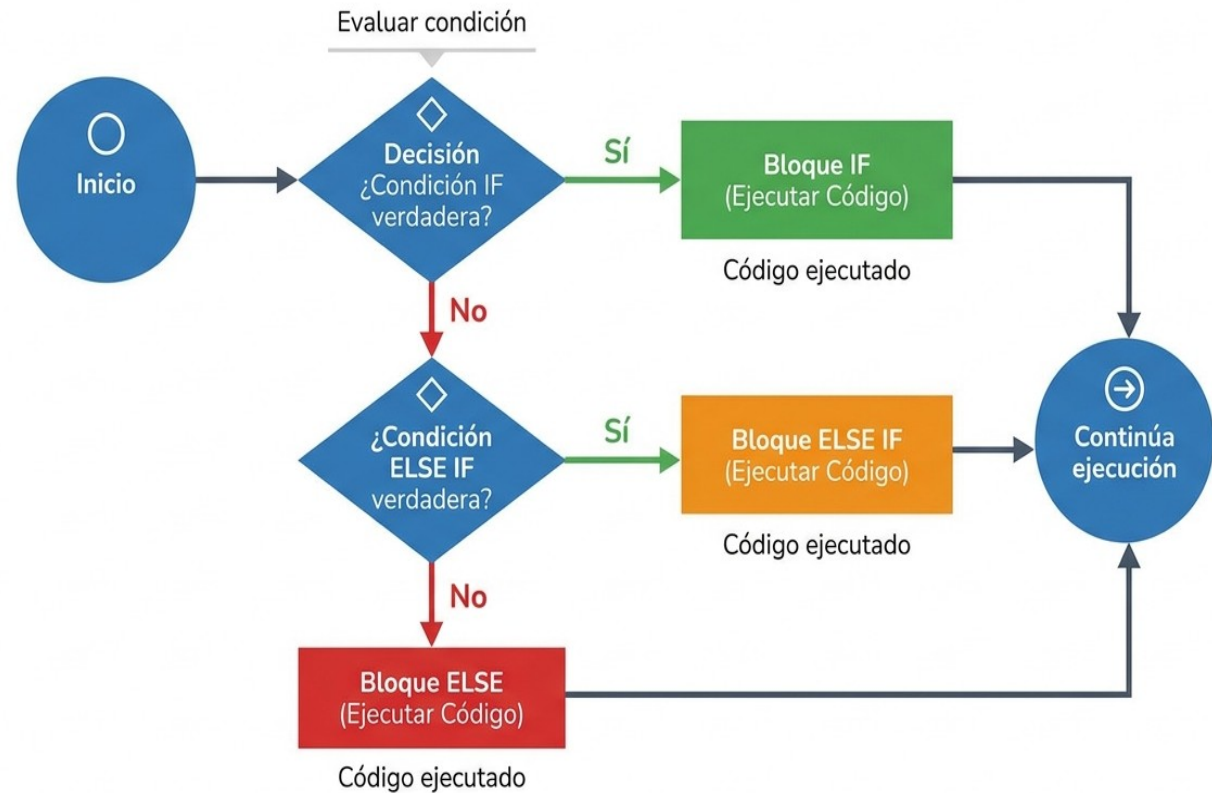
- Introducción

Rara vez todos los escenarios de prueba son iguales. El bloque IF / ELSE IF / ELSE te permite ejecutar keywords distintas según una condición.

¿Qué es un Bloque Condicional en RF?

- ▶ Permite ejecutar keywords solo si una condición es verdadera.
- ▶ Evalúa expresiones booleanas en orden: IF → ELSE IF → ELSE.
- ▶ Reacciona al estado real del sistema bajo prueba.
- ▶ Disponible de forma nativa desde Robot Framework 4.0.

DIAGRAMA DE FLUJO: ESTRUCTURA CONDICIONAL IF / ELSE IF / ELSE



Sintaxis Nativa del Bloque IF (RF 4.0+)

```
# Sintaxis básica del bloque IF nativo (RF 4.0+)
IF    ${condicion}
    Log    La condición es verdadera
END

# Bloque IF con ELSE IF y ELSE
IF    '${estado}' == 'activo'
    Log    El usuario está activo
ELSE IF    '${estado}' == 'suspendido'
    Log    El usuario está suspendido
ELSE
    Log    Estado desconocido: ${estado}
END
```

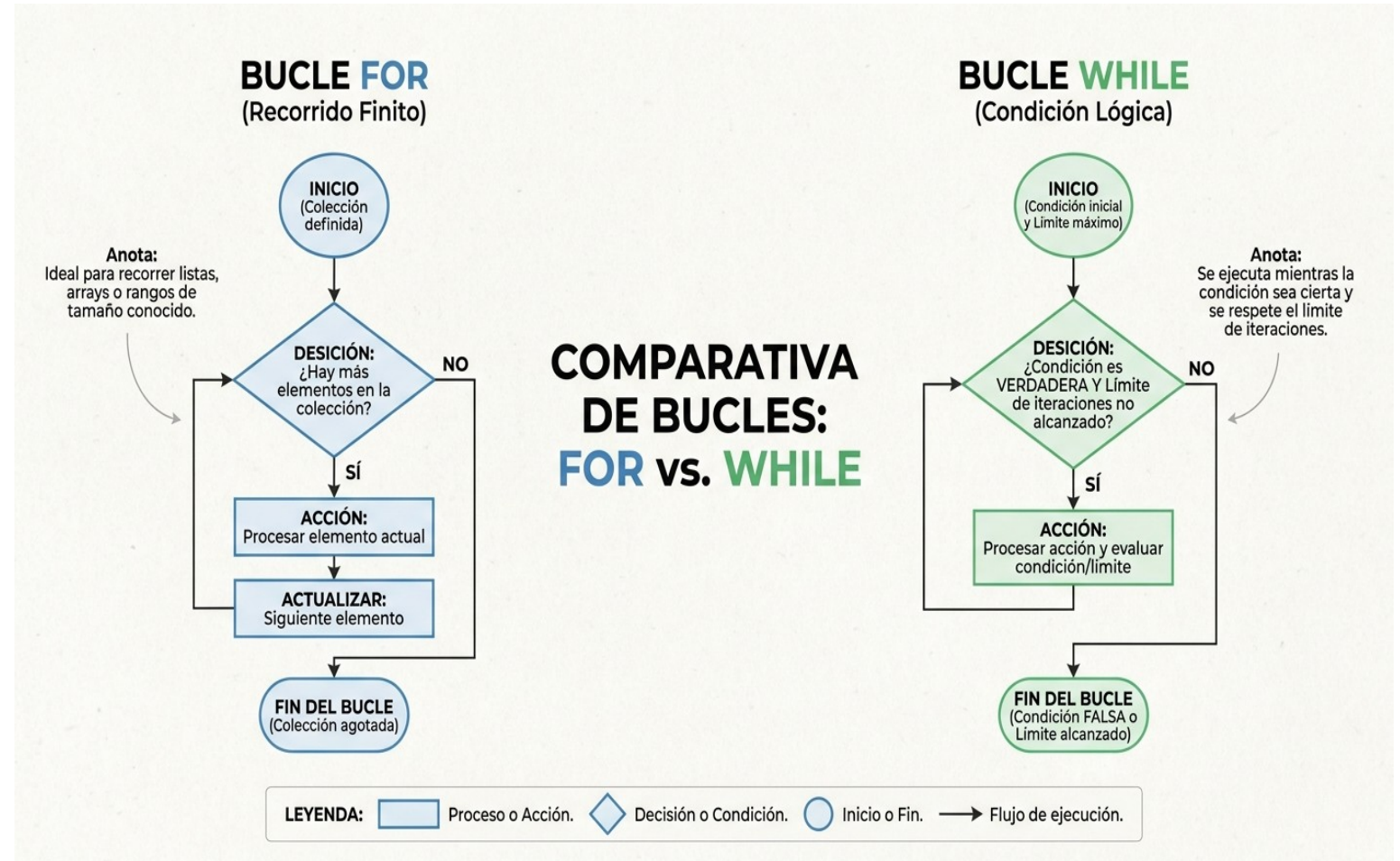
3.2: FOR / WHILE: Iteración sobre Listas, Rangos y Condiciones

- Introducción

Cuando trabajas con varios datos a la vez (una lista de clientes, de consumos, de endpoints), FOR te permite repetir una verificación para cada uno sin duplicar código.

Estructura General de los Bucles

- FOR itera sobre listas, rangos o pares clave-valor.
- WHILE repite mientras una condición sea verdadera.
- Ambos admiten BREAK y CONTINUE para control fino.
- Los bucles pueden anidarse; minimizar la profundidad.



FOR: IN, IN RANGE e IN ENUMERATE

*** Test Cases ***

Iterar Lista

```
@{usuarios}    Create List    ana    luis    pedro
FOR    ${usuario}    IN    @{usuarios}
    Log    Procesando: ${usuario}
END
```

Iterar Rango Con Paso

```
FOR    ${n}    IN RANGE    0    10    2
    Log    Valor par: ${n}
END
```

Iterar Con Indice

```
@{frutas}    Create List    manzana    pera    uva
FOR    ${i}    ${fruta}    IN ENUMERATE    @{frutas}    start=1
    Log    ${i}. ${fruta}
END
```

3.3: Asserts y Fallas Controladas: Should Be, Run Keyword And Expect Error

- Introducción

Validar que el sistema falla CORRECTAMENTE es tan importante como validar que funciona. Run Keyword And Expect Error confirma que un error esperado ocurre, sin fallar el test.

Detalles Técnicos de las Aserciones Should Be

- ▶ Pertenecen a la biblioteca BuiltIn: se cargan automáticamente sin importación adicional.
- ▶ El argumento msg= personaliza el mensaje de error visible en reportes y pipelines CI/CD.
- ▶ Should Be Equal hace comparación estricta de tipo y valor: `5 != "5"`.
- ▶ Should Be Equal As Strings y As Numbers normalizan el tipo antes de comparar.
- ▶ Should Be True acepta cualquier expresión Python válida para condiciones compuestas.
- ▶ Una aserción fallida detiene el test case inmediatamente para evitar pasos sobre estado inválido.

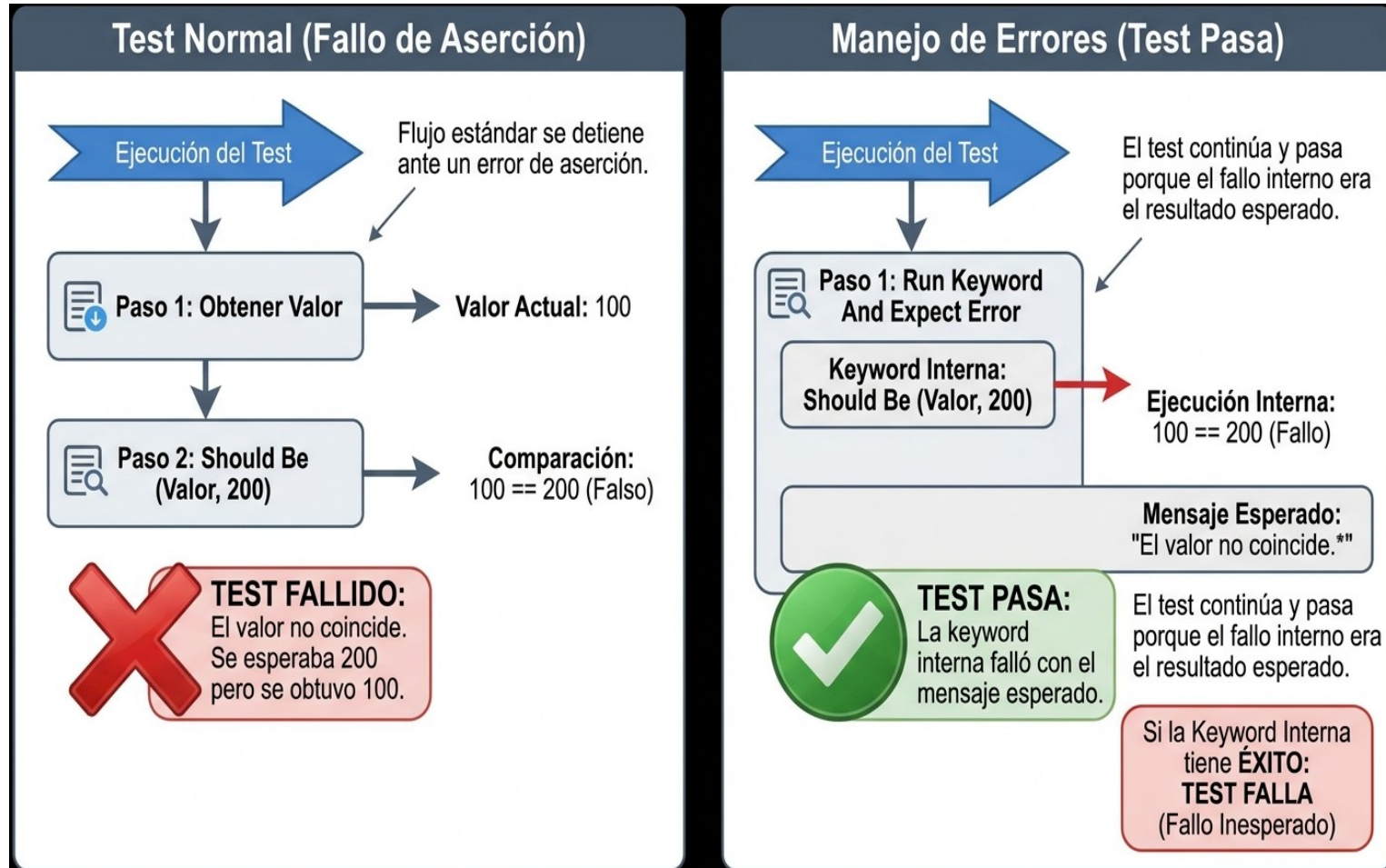
Run Keyword And Expect Error: Concepto

- ▶ Run Keyword And Expect Error pertenece a BuiltIn e invierte la lógica habitual.
- ▶ Espera que la keyword interna falle; si falla con el mensaje esperado, el test pasa.
- ▶ Si la keyword interna tiene éxito (no lanza error), el test falla.
- ▶ Fundamental para probar comportamientos de rechazo y validaciones de error.
- ▶ Cubre: validaciones de formulario, excepciones de negocio, errores de API.
- ▶ El valor de retorno es el mensaje de error real, útil para inspecciones adicionales.

Run Keyword And Expect Error: Patrones de Error

- ▶ Firma: `${msg}= Run Keyword And Expect Error patrón Keyword args`.
- ▶ Primer argumento: patrón del mensaje de error esperado.
- ▶ Texto exacto: el patrón debe coincidir completamente con el error real.
- ▶ Comodines glob (*): por defecto, `*no permitido*` acepta cualquier prefijo/sufijo.
- ▶ Prefijo REGEXP:: permite usar expresiones regulares para patrones complejos.
- ▶ Si el patrón no coincide con el error real, la keyword también falla.

Fallas Controladas vs. No Controladas



- ▶ Falla no controlada: detiene el test inesperadamente → FAIL.
- ▶ Falla controlada: capturada con Run Keyword And Expect Error → PASS.
- ▶ El error esperado forma parte del diseño intencional de la prueba.
- ▶ Sin captura, los escenarios de rechazo quedan sin cobertura real.

3.4: Continue On Failure, Run Keyword And Continue On Failure y Recuperación

- Introducción

Por defecto, Robot Framework detiene un test en el primer fallo (fail fast). Run Keyword And Continue On Failure te deja ver TODOS los errores de un test, no solo el primero.

Comportamiento Predeterminado: Fail Fast

- ▶ Robot Framework sigue un modelo de fail fast: detiene el caso al primer fallo.
- ▶ Cuando una keyword lanza excepción, el caso se marca como FAIL inmediatamente.
- ▶ Las keywords siguientes dentro del caso no se ejecutan.
- ▶ En escenarios complejos, esto impide recopilar evidencias completas.
- ▶ Ej: validar 10 campos de API — un fallo en el 2do oculta los 8 restantes.
- ▶ Esto obliga a ejecutar la suite varias veces para descubrir todos los defectos.
- ▶ El modo Continue On Failure resuelve este problema permitiendo continuar.

Continue On Failure: Ejemplo Básico

```
*** Test Cases ***
# Sin Continue On Failure: un fallo detiene todo
Validar Campos De Respuesta API
    Should Be Equal As Strings    ${status_code}    200
    Should Contain    ${body}    "usuario"
    Should Not Be Empty    ${body}[token]

# Con Continue On Failure: todos los campos son evaluados
Validar Campos De Respuesta API Con Continuacion
    Run Keyword And Continue On Failure
    ...    Should Be Equal As Strings    ${status_code}    200
    Run Keyword And Continue On Failure
    ...    Should Contain    ${body}    "usuario"
    Run Keyword And Continue On Failure
    ...    Should Not Be Empty    ${body}[token]
Log    Todas las validaciones fueron evaluadas
```

Run Keyword And Continue On Failure: Concepto

- ▶ Keyword de la biblioteca BuiltIn que actúa como envoltorio (wrapper).
- ▶ Ejecuta la keyword indicada; captura la excepción internamente si falla.
- ▶ Registra el error en el log con nivel FAIL y retorna el control al caso.
- ▶ El caso de prueba continúa con la siguiente instrucción normalmente.
- ▶ Ideal para validaciones múltiples e independientes en un mismo caso.
- ▶ Útil para verificar varios elementos de UI o campos de datos simultáneamente.
- ▶ No silencia el fallo: el caso sigue marcándose como FAIL al finalizar.

Resumen del capítulo

- El bloque IF / ELSE IF / ELSE nativo de Robot Framework (disponible desde la versión 4.0) permite ejecutar keywords de forma condicional con una sintaxis clara, indentada y cerrada con END.
- Las condiciones se evalúan como expresiones Python, lo que permite usar operadores de comparación (==, !=, >, <) y lógicos (and, or, not).
- Se pueden encadenar múltiples ramas ELSE IF para cubrir distintos escenarios, y el bloque ELSE actúa como caso por defecto cuando ninguna condición anterior se cumple.
- El enfoque heredado con Run Keyword If sigue siendo funcional pero resulta menos legible; se recomienda el bloque nativo en proyectos nuevos o al refactorizar suites existentes.
- Los bloques condicionales pueden usarse tanto dentro de keywords personalizadas como directamente en el cuerpo de un test case, lo que los hace versátiles para distintos niveles de abstracción.



Resumen del capítulo

- Anidar bloques IF es posible, pero conviene extraer la lógica compleja a keywords con nombres descriptivos para preservar la legibilidad de la suite.
- El bucle FOR recorre colecciones con IN, rangos numéricos con IN RANGE, pares enumerados con IN ENUMERATE y combinaciones paralelas con IN ZIP.
- El bucle WHILE evalúa una condición booleana en cada iteración; incluye un límite de seguridad configurable con el argumento limit para prevenir bucles infinitos.
- BREAK interrumpe el bucle por completo; CONTINUE omite el resto del bloque actual y avanza a la siguiente iteración.
- Ambos bucles se cierran con END y pueden anidarse, aunque se recomienda mantener la profundidad de anidamiento al mínimo para preservar la legibilidad.



Práctica 5: Tests con lógica condicional y bucles de datos

Objetivo:

Crear tests que usen IF/ELSE para clasificar consumos y FOR para recorrer una lista de clientes validando sus límites.

Guía completa: [robfram-guide/Capitulo03/Practica5.md](#)



Práctica 6: Suite robusta con manejo de fallas y recuperación

Objetivo:

Diseñar una suite que use Continue On Failure, Expect Error e Ignore Error para manejar fallas de forma controlada.

Guía completa: [robfram-guide/Capitulo03/Practica6.md](#)



Referencias Bibliográficas

Capítulo 3: Control de Flujo y Validaciones

- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#if-else-if-else-structure>
- ▶ <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html#Run%20Keyword%20If>
- ▶ <https://github.com/robotframework/robotframework/blob/master/doc/releasenotes/rf-4.0.rst>
- ▶ https://docs.robotframework.org/docs/getting_started/testing
- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#for-loops>